

Lecture 6 (Part 2)

KNN, Hyperparameters, and Feature Engineering

Matt McDonald

What is KNN?

K-nearest neighbors (KNN) predicts by looking at the k closest training observations.

For classification:

- each neighbor “votes”
- prediction can be a class label
- or a probability estimate from the vote share

KNN: a geometric mental model

Think of each observation as a point in feature space:

- 2 features → a 2D plane
- 20 features → same idea, harder to visualize

KNN says:

“Look around this new point. What do its neighbors do?”

Distance is the whole game

KNN depends on:

- a distance metric (usually Euclidean)
- how features are scaled
- which features are included

So KNN is *as much about features as it is about the algorithm*.

The k dial (flexibility)

- small k (1–5):
 - very flexible boundary
 - can “memorize” training data
 - high variance
- large k (50–200):
 - smooths noise
 - higher bias
 - more stable

Hyperparameter (new term)

A **hyperparameter** is a model setting we choose *before fitting* that controls model complexity.

KNN's hyperparameter is:

- k

Examples you'll see soon:

- ridge/lasso: λ
- polynomial regression: degree
- tree models: max depth (later)

Parameter vs hyperparameter

- **Parameters** are learned from training data (e.g., logistic regression coefficients)
- **Hyperparameters** are chosen by us (typically using CV)

Today:

- We will choose k using cross-validation.

The **Caravan** dataset

What it is:

- An insurance marketing dataset
- Outcome: **did the customer buy a caravan insurance policy?** (**Purchase** Yes/No)
- Predictors: many customer features (demographics / prior products / etc.)

What makes it a good teaching dataset:

- the outcome is **rare**
- signal is **weak** and spread across many features

Why it's useful for KNN

KNN is a distance-based method, so we get to see:

- scaling really matters
- small k can look great in-sample but degrade out-of-sample
- CV gives a principled way to choose k

Quick reminder: why accuracy can mislead

If the event is rare:

- predicting “No” every time can still give high accuracy
- better: AUC, sensitivity/specificity, precision/recall (depending on goal)

Scaling: why it matters for KNN

KNN uses distances.

If one variable has a bigger numeric range:

- it dominates distance
- neighbors are chosen based on units, not information

So we often **center and scale**.

Center + scale

For feature x :

- center: $x - \bar{x}_{train}$
- scale: $\frac{x - \bar{x}_{train}}{s_{train}}$

We do this using **training statistics only**.

Leakage warning: scaling inside resampling

Correct approach:

- compute mean/SD on training fold
- apply to validation fold

Incorrect approach:

- compute mean/SD on the full dataset
- (this leaks information into the training fold)

In-sample vs out-of-sample (the trap)

If you evaluate on training data:

- small k can look great
- you are measuring memorization, not generalization

If you evaluate on held-out data (CV/test):

- small k often degrades
- there is usually a "sweet spot" k^*

Why small k can overfit

With $k = 1$:

- each point is its own best neighbor
- decision boundary can become extremely jagged
- noise points can dominate predictions locally

When KNN struggles

KNN often struggles when:

- features are high-dimensional (curse of dimensionality)
- signal is global/linear (simple models win)
- you have lots of irrelevant features
- missingness is common (distance becomes messy)

Curse of dimensionality

As the number of features grows:

- “nearest neighbors” become less distinct
- distances bunch together
- you often need more data to get stable neighborhoods

Feature engineering (and sometimes feature selection) helps.

How we'll tune k (workflow)

1. Split data into Train/Test (test is final exam)
2. Create CV folds inside training
3. For each k in a grid:
 - scale inside each fold
 - fit/predict KNN
 - compute AUC on validation fold
4. Choose k with best mean CV AUC
5. Evaluate once on test

Why a grid (not calculus)

For KNN:

- there's no simple analytic solution for "best k "
- grid search is easy and transparent
- you can try a sensible range (small to big)

Example grid:

- $k \in \{1, 3, 5, 11, 25, 51, 101, 201\}$

What we expect to see

A typical pattern:

- training AUC decreases as k increases (less memorization)
- validation AUC improves then plateaus, then can decline
- the best k is rarely 1

CV output you should show

For each k :

- mean CV AUC
- SD across folds

This provides:

- a point estimate
- a sense of stability

Feature engineering: what is it?

Feature engineering is changing the representation of inputs to make learning easier.

Two goals:

- capture the relationship more directly
- make the algorithm behave well (stability, robustness)

Feature engineering is algorithm-dependent

Different algorithms are sensitive to different things:

- KNN: scaling, irrelevant features, missing data
- linear/logistic regression: outliers, nonlinearity, multicollinearity
- tree models: less sensitive to scaling, can handle nonlinearity, but can overfit
- many methods: class imbalance affects decision thresholds and metrics

Feature engineering toolbox (overview)

Common operations:

- scaling / standardizing
- transforming (log, sqrt, winsorize)
- encoding categorical variables (dummies)
- interactions and ratios
- binning
- nonlinear basis expansions (polynomials, splines)
- missing data imputation
- handling class imbalance (over/under-sampling)

Scaling & normalization

Used when:

- distance-based methods (KNN)
- regularized regression (ridge/lasso)
- gradient-based models

Not usually needed for:

- decision trees / random forests

Transformations (practical finance examples)

Transformations can:

- reduce skew (log income, log balance)
- tame outliers (winsorize)
- stabilize variance

Examples in finance:

- log market cap
- log spreads
- z-scores of returns

Encoding categorical variables

Many algorithms require numeric input:

- dummy variables (one-hot)
- careful about “baseline” category interpretation

Some models handle categories natively (not our focus today).

Interactions (when simple additivity fails)

If the effect of one variable depends on another:

- include an interaction term
- or create a ratio (e.g., balance / income)

Interpretation:

- “the slope changes depending on the other feature”

Nonlinearity as feature engineering

You can keep a simple model but add nonlinear features:

- polynomial terms
- splines (basis functions)
- smooth terms (GAMs)

This is a bridge to Lecture 7.

Missing data: feature engineering vs sidelining a model

Some algorithms can't handle missing values directly.

Options:

- imputation (mean/median, regression, KNN imputation)
- missingness indicators ("was missing" flag)
- model choices that handle missingness better

Key point:

Sometimes you adapt the data, not the algorithm.

Class imbalance (why it matters)

Imbalance affects:

- accuracy (misleading)
- threshold-based decisions
- probability calibration
- minority-class recall/precision

What you can do (feature engineering / data engineering):

- over-sample minority class
- under-sample majority class
- use class weights (algorithm-dependent)
- change the threshold to match costs

Feature engineering inside resampling (rule)

If the transformation uses data statistics:

- scaling
- imputation
- PCA
- feature selection

Then it must be learned on the training fold and applied to validation fold.

Otherwise:

- leakage
- optimistic CV

Live Coding

What we just did (and why it felt like “work”)

We ran v-fold cross-validation by hand:

- created folds
- scaled inside each fold (no leakage)
- fit the model repeatedly
- scored each fold and summarized results

That's a lot of moving parts — *on purpose* — so you could see what's happening.

Why frameworks exist

In real projects, this gets repetitive fast:

- preprocessing inside resampling
- repeated fitting + scoring
- keeping everything organized
- avoiding subtle leakage bugs

So people use workflows that automate the plumbing.

tidymodels (big picture)

tidymodels is a suite of R packages designed to make modeling workflows easier and safer.

We already used (a piece of it):

- `rsample` (splits + folds)
- `yardstick` (metrics)

Other pieces help with:

- preprocessing
- model interfaces
- tuning
- end-to-end workflows

What tidymodels would automate (conceptually)

If we leaned into tidymodels, it can:

- apply preprocessing inside folds automatically
- tune hyperparameters without writing loops
- keep results organized in a consistent structure
- reduce the chance of leakage

But...

Why we're *not* going deep today

tidymodels is powerful, but it's a whole thing:

- new vocabulary
- new object types
- a different workflow style

For this course (Part 1), we're prioritizing:

- clear statistical ideas
- base R models when possible
- minimal "framework overhead"

If you're interested...

If you want to explore later:

- search: "tidymodels getting started"
- focus on: `rsample`, `recipes`, `parsnip`, `workflows`, `tune`, `yardstick`

Optional goal:

- replicate today's demo using a tidymodels workflow
- compare how much code it saves

Wrap-up (Part 2)

- KNN is simple but feature-dependent
- k is a hyperparameter (chosen by CV)
- scaling is mandatory for distance-based methods
- feature engineering is a toolbox to make models work better (and fairly)